

Sofranet - Proje Raporu

Ders: CENG 382 - Web Programming **Dönem:** 2025-2026 Bahar **Öğrenci:** Yusuf Şimşek (202138008) **Bölüm:** Yönetim Bilişim Sistemleri **Üniversite:** Çankaya Üniversitesi

1. Proje Tanımı

Sofranet, kullanıcıların yakınlarındaki yerel restoranlardan yemek sipariş edebildiği, restoran sahiplerinin (caterer) kendi menülerini yönetebildiği ve sistem yöneticilerinin platformu denetleyebildiği bir full-stack catering platformudur. Proje, ders kapsamında istenen tüm fonksiyonel gereksinimleri ve bonus 2FA kalemını karşılayacak şekilde tasarlanmış, ASP.NET Core 8 MVC mimarisi üzerine kurulmuştur.

Platform üç farklı kullanıcı rolünü destekler:

- User (Müşteri):** Menüleri tarar, sepet oluşturur, sipariş verir, tamamlanmış siparişlere değerlendirme yapar
- Caterer (Restoran):** Menü itemları ve customization seçenekleri tanımlar, gelen siparişleri yönetir
- Admin (Yönetici):** Sistemdeki tüm kullanıcıları, siparişleri ve aktivite log'larını denetler

2. Sistem Mimarisi

Uygulama, ASP.NET Core MVC desenine uygun şekilde geliştirilmiştir. Server-side rendering kullanılmıştır; Razor view engine ile HTML çıktısı sunucuda üretilir.

Katmanlar

- Controllers:** HTTP endpoint'lerini barındırır, model binding ve action result döner
- Models:** EF Core entity'leri ve view-spesifik ViewModels
- Views:** Razor (.cshtml) templates
- Services:** Email, PDF, Maps, Logging için dış servis entegrasyonları (DI ile)
- Data:** ApplicationDbContext (Identity'den türeyen) ve seed
- Helpers:** Order enrichment, file upload, session helpers
- wwwroot:** Static dosyalar (CSS, JS, images, uploads)

İstek Akışı

```
HTTP Request
  ↓
Program.cs middleware pipeline
(UseStaticFiles → UseRouting → UseSession → UseAuthentication → UseAuthorization)
  ↓
Controller (model binding, [Authorize] kontrolü)
  ↓
DbContext / Services
  ↓
View (Razor render) → HTTP Response
```

3. Teknoloji Stack'i

- **.NET 8** - LTS sürüm
- **ASP.NET Core 8 MVC** - web framework
- **Entity Framework Core 8** - ORM, Code-First migration ile
- **ASP.NET Core Identity** - kullanıcı yönetimi, rol bazlı yetkilendirme, 2FA
- **SQLite** - başlangıçta PostgreSQL planlanmıştı, geliştirme makinesinde PostgreSQL kurulu olmadığı için SQLite'a geçildi (provider değişimi tek bir Program.cs satırı). Production'da connection string ile Postgres'e dönülebilir.
- **MailKit** - SMTP üzerinden e-posta gönderimi (sipariş bildirimi, 2FA kodu)
- **QuestPDF** (Community license) - PDF üretimi için fluent API
- **Google Maps API** - Geocoding, Distance Matrix, Maps JavaScript (API key olmadığında Haversine fallback)
- **Bootstrap 5** - responsive grid + form/utility component'ları
- **Google Fonts Poppins** - tipografi

4. Kullanıcı Roller ve Yetkilendirme

Identity'nin RoleManager'ı ile üç rol tanımlandı: Admin, Caterer, User. Controller seviyesinde `[Authorize(Roles = "...")]` attribute'leri ile koruma sağlandı.

Rol	Erişebildiği Alanlar
Admin	/Admin/* (dashboard, kullanıcılar, restoranlar, siparişler, loglar)
Caterer	/Caterer/* (kendi menüleri, customization, siparişleri)
User	/Cart, /Order, /Rating, /User
Herkes	/, /Menu (anonim browsing), /Account/Register, /Account/Login

Kayıt formunda kullanıcı rol seçer (User veya Caterer). Admin sadece seed yoluyla oluşturulur. Login sonrası rol bazlı redirect uygulanır: Admin → /Admin, Caterer → /Caterer, User → /User.

5. Veritabanı Şeması

Identity tabloları (`AspNetUsers` , `AspNetRoles` , `AspNetUserRoles` , vb.) dışında 10 domain tablosu var:

- `MenuItems` - caterer'in menü ürünleri (Name, Price, ImagePath, IsAvailable, CatererId FK)
- `OptionGroups` - menü item'a bağlı seçim grupları (örn: Sos, Boyut)
- `MenuOptions` - bir grup içindeki tek seçenek (Name, ExtraPrice)
- `RemovableIngredients` - çıkartılabilir malzemeler (örn: Soğan, Maydanoz)
- `Orders` - kullanıcı siparişleri (UserId, CatererId, TotalAmount, Status, DeliveryAddress)
- `OrderItems` - sipariş içindeki her ürün satırı (Quantity, ItemPrice snapshot, CustomizationExtra)
- `OrderItemOptions` - sipariş item için seçilen seçenekler
- `OrderItemRemovals` - sipariş item için çıkartılan malzemeler
- `Ratings` - sipariş başına bir değerlendirme (OrderId UNIQUE, MenuRating + CatererRating + Comment)
- `LogEntries` - sistem audit log (Action, Details, IPAddress, UserId nullable, CreatedAt)

Önemli ilişki detayları: - Order → User ve Order → Caterer (her ikisi de ApplicationUser FK) için `onDelete(DeleteBehavior.Restrict)` cycle önlemek için - Rating.OrderId UNIQUE INDEX ile bir siparişe tek değerlendirme garantisi - LogEntry.UserId nullable ve `onDelete(SetNull)` - kullanıcı silinse bile log korunur - OrderItem snapshot pattern: ItemPrice sipariş anındaki fiyatı tutar, MenuItem güncellense de değişmez

EF Core Code-First migration tek `InitialCreate` adı altında. Migration prod'da otomatik uygulanır (SeedData.InitializeAsync içinde `MigrateAsync`).

6. Uygulanan Özellikler

Aşağıdaki tablo, ders rubric'indeki kalemler ile koddaki karşılıklarını eşler.

Kalem	Puan	Karşılık
Role structure & authorization	10	Identity + [Authorize(Roles)] her controller'da
Login auth system	10	AccountController, custom Login/Register view'ları
Role-based dashboards	10	/Admin, /Caterer, /User Index — gerçek stat'ler
Menu system (CRUD + customization)	12	CatererController Menu+Customize, OptionGroup/MenuOption/Removable
Cart & order flow	8	CartController (session), OrderController.Create (DB transaction)
Google Maps integration	10	Geocoding, Distance Matrix, JS Maps + Haversine fallback
Rating system	8	RatingController + 2-star UI (yemek + restoran)
Email system	6	EmailService (MailKit), sipariş & 2FA bildirimleri
Receipt PDF	5	PdfService.GenerateReceipt (QuestPDF)
Agreement PDF	5	PdfService.GenerateAgreement (7 maddelik sözleşme)
Logging system	6	LogService, her kritik action LogEntry'e yazılır
Admin logging page	4	/Admin/Logs - filter (action, user, ip, tarih), pagination
Filter & pagination	3	Menu, MyOrders, Caterer/Orders, Admin tabloları
Payment simulation	3	/Cart/Checkout - 4242 OK, sonu 0000 reject
UI/UX & branding	5	Sıcak renk paleti, responsive, animasyonlar, 404 sayfası
Report, scripts, demo	5	Bu dosya + README.md + git history
Bonus: 2FA	+10	Email tabanlı, Identity'nin built-in mekanizması

WebRTC live call (+20) atlandı — hızlı geliştirme akışında kırılğan olduğu için dahil edilmedi.

7. Önemli Teknik Kararlar

PostgreSQL → SQLite geçişi

Başlangıç planında Npgsql provider ile Postgres kullanmak vardı, ancak geliştirme makinesinde Postgres kurulu olmadığı için SQLite'a geçildi. Production veritabanı değişimi sadece şu adımlardan geçer:

1. `dotnet remove package Microsoft.EntityFrameworkCore.Sqlite`

2. `dotnet add package Npgsql.EntityFrameworkCore.PostgreSQL`
3. `Program.cs` 'de `UseSqlite` → `UseNpgsql`
4. `appsettings.json` connection string güncelle
5. Migration regenerate

SQLite ile öğrendiğim ilginç bir nokta: SQLite, `decimal` üzerinde server-side `SUM()` desteklemiyor. Tüm `SumAsync(o => o.TotalAmount)` çağrılarını `Select().ToListAsync().Sum()` (in-memory aggregate) ile değiştirdim.

"action" parametre çakışması

`AdminController.Logs(string? action, ...)` ile dropdown filtresi yazmıştım. Filtreleme çalışmadı çünkü ASP.NET Core MVC'de "action" parameter ismi route value'sundan otomatik bind ediliyor — model binder `action = "Logs"` (route value) atıyor, query string'i göz ardı ediyor. Parametre ismini `actionType` olarak değiştirince çözüldü. Aynı durum "controller" için de geçerli.

Session-based cart

Kullanıcı sepeti DB yerine session'da JSON olarak saklanıyor. Avantajları:

- Sipariş tamamlanana kadar DB'de yer kaplamaz
- Authentication kaybolunca sepet de uçar (istenen davranış)
- Genişletilmesi kolay (örn: misafir kullanıcı için)

Sepet üzerine option seçimi yapıldıkça gerçek menu/option ID'leri DB'den valide edilir, sahte ID gönderilmesi engellenir.

Farklı caterer onay flow'u

Kullanıcının sepetinde A restoranından ürün varken B restoranından ürün eklemeye çalışırsa:

1. Yeni item `cart_pending` session key'ine kaydedilir, ana sepete eklenmez
2. /Cart sayfasında banner çıkar: "Sepetinde başka bir restoranın ürünleri var..."
3. Kullanıcı "Sepeti Temizle ve Ekle" veya "Vazgeç" seçer

Payment simulation

Gerçek payment gateway yok. Kart numarasının son 4 hanesi `0000` ise reddedilir, `4242...4242` test kartı her zaman onaylanır, diğerleri OK. Onay sonrası session'a `payment_approved=true` yazılır, kullanıcı `/Order/Create` POST'una basana kadar bir adım atılması gerekir.

Order creation transaction

`OrderController.Create` action'ı `_db.Database.BeginTransactionAsync()` ile sarılı. Sipariş + items + options + removals atomik insert. Email gönderimi transaction commit sonrası, try-catch ile sarılı — SMTP fail olsa bile sipariş kaybolmaz.

2FA implementasyonu

Identity'nin built-in `TwoFactorEnabled` property'si kullanıldı. Login akışı:

1. `PasswordSignInAsync` → `RequiresTwoFactor` döner ise partial sign-in cookie set edilir
2. `GenerateTwoFactorTokenAsync("Email")` ile 6 haneli kod üretilir, `EmailService` ile gönderilir
3. Kullanıcı `/Account/LoginWithTwoFactor` sayfasına yönlendirilir
4. `TwoFactorSignInAsync("Email", code, ...)` ile kod doğrulanır, full sign-in tamamlanır

Token lifespan default 1 günden 5 dakikaya çekildi (`DataProtectionTokenProviderOptions`).

8. Seed Verisi

Uygulama ilk çalıştığında `Data/SeedData.cs` aşağıdaki verileri üretir:

- 3 rol: Admin, Caterer, User
- 1 admin: admin@sofranet.com / Admin123
- 3 caterer: Lezzet Mutfağı (Kadıköy), Mahalle Sofrası (Beyoğlu), Ankara Gurme (Çankaya)
- 5 user: Ali, Ayşe, Mehmet, Zeynep, Can (hepsinin şifresi: User123)
- 11 menu item Türk mutfağından (Lahmacun, Adana Dürüm, Künefe, İskender, Kuru Fasulye, Manti, vb.)
- Her menüye 0-2 option group + 0-3 removable ingredient

Seed idempotent: tekrar çalıştırılırsa duplicate üretmez (`if (!await db.MenuItems.AnyAsync())`).

9. Kurulum

`README.md` dosyasındaki adımlar takip edilebilir. Özet:

```
git clone <repo>
cd Sofranet
dotnet restore
dotnet ef database update
dotnet run
```

<http://localhost:5099> üzerinden erişilir. SMTP ve Google Maps API key boş bırakılabilir; uygulama bu durumda graceful degradation yapar (email log'a yazılır, harita fallback mesajı gösterir, Distance Matrix yerine Haversine kullanılır).

10. Geliştirme Süreci

Proje, 12 ana commit ile geliştirildi. Her commit mantıksal bir birim (initial setup → auth → menü → cart → ödeme → email/PDF → rating + dashboardlar → admin → 2FA → polish → finalize). git log üzerinden iterasyonlar takip edilebilir.

Test stratejisi: her commit sonrası temel smoke test (login, kritik action'lar) cURL ile yapıldı. Unit/integration test eklenmedi — ödev kapsamı dışında.

11. Eksiklikler ve İleride Geliştirilebilecek Yönler

- Order list'lerinde sort seçeneği yok (sadece tarih azalan).
- Caterer dashboard'da haftalık/aylık trend grafiği yok.
- Image upload sadece local disk'e — production için S3 veya benzeri object storage gerekir.
- Session memory'de — multi-instance deploy için Redis veya distributed cache gerekir.
- Email rate limiting yok — 2FA için brute force koruması ek bir middleware ile yapılabilir.
- SignalR ile gerçek zamanlı sipariş bildirimi yok (caterer panelinin polling yapması gerekiyor).
- Frontend i18n yok — hardcoded Türkçe.

12. Sonuç

Sofranet, ders rubric'inde istenen tüm zorunlu kalemleri ve bonus 2FA özelliğini karşılayan, çalışan bir MVC uygulaması olarak teslim edildi. Temel akışlar (kayıt, login, menü browse, sipariş, ödeme simülasyonu, PDF üretimi, rating, admin denetimi) uçtan uca test edildi. SQLite'in bazı sınırları (decimal aggregate desteklememesi) ve MVC reserved route parameter tuzağı gibi geliştirme sırasında karşılaşılan teknik problemler çözüldü.

Proje, modern ASP.NET Core ekosisteminin (Identity, EF Core, MailKit, QuestPDF) pratik kullanımını gösterir niteliktedir.